Course Cards Filter — JavaScript Walkthrough

*A section-by-section explanation of the client-side filtering logic for the **cmp-coursecards** AEM component.*

1. The Outer Wrapper (IIFE)

```
(function() {
    'use strict';
    // ...
})();
```

This is an **IIFE** (Immediately Invoked Function Expression). The `(function(){...})()` pattern defines a function and runs it right away. Variables declared inside stay private and do not leak into the global `window` scope. `'use strict'` turns on stricter JavaScript rules, which catches silent bugs like assigning to undeclared variables.

2. Main Function Declaration

```
function initCourseCardsFilter() {
    var components = document.querySelectorAll('.cmp-coursecards');
    if (!components.length) return;
```

`querySelectorAll` finds every element on the page with the class **cmp-coursecards** and returns a NodeList. If none exist on this page, the function exits immediately. The reason it looks for *multiple* components is that in AEM the same component can be dropped on a page more than once — each instance needs its own independent filter state.

3. Looping Over Each Component Instance

```
components.forEach(function(component) {
    var filterButtons = component.querySelectorAll('.cmp-coursecards__filter');
    var limitSelect   = component.querySelector('#course-limit-select');
    var cards         = component.querySelectorAll('.cmp-coursecards__card');
    var noMatch       = component.querySelector('.cmp-coursecards__no-match');
    var countNum      = component.querySelector('.cmp-coursecards__count-num');
```

For each component found, the script grabs the pieces inside it: the category filter buttons, the limit dropdown, every course card, the no-match message, and the visible-count display. All lookups use **component.querySelector(...)** — scoped to that one instance — so two instances of the widget do not interfere with each other. If the instance has no cards, the iteration is skipped.

4. State Variables

```
var currentCategory = 'all';
var currentLimit = limitSelect ? limitSelect.value : 'all';
```

Two pieces of state are tracked per component: **currentCategory** (which filter is selected, defaulting to *'all'*) and **currentLimit** (how many cards to show, defaulting to whatever the dropdown currently says). These live inside the `forEach` callback, so each component instance gets its own copy — that is the closure trick that keeps instances independent.

5. The applyFilters Function — Core Logic

```
function applyFilters() {
```

```
    var limit = (currentLimit === 'all') ? cards.length
                                         : parseInt(currentLimit, 10);
    var visibleCount = 0;
```

First, figure out the numeric limit. If the user picked *'all'*, the limit is the total card count. Otherwise the string value (e.g. `'5'`) is converted to a number. **visibleCount** tracks how many cards have actually been shown.

```
cards.forEach(function(card) {
    var cardCategory = card.getAttribute('data-category') || '';
    var categoryMatches = (currentCategory === 'all') ||
                          (cardCategory === currentCategory);
```

For each card, read its **data-category** attribute (set in HTL like `data-category="java"`). A card matches if either the user selected "All", or the card's category equals the currently selected one.

```
    if (categoryMatches && visibleCount < limit) {
        card.classList.remove('cmp-coursecards__card--hidden');
        card.style.display = '';
        visibleCount++;
    } else {
        card.classList.add('cmp-coursecards__card--hidden');
        card.style.display = 'none';
    }
});
```

Two conditions must hold to show a card: it matches the category, AND the maximum has not been reached yet. If both pass, the hidden class is removed, inline `display` is cleared, and the counter goes up. Otherwise the card is hidden. (Side note: doing both `classList.add` and `style.display = 'none'` is redundant — either alone would hide it.)

```
    if (countNum) {
        countNum.textContent = visibleCount;
    }
    if (noMatch) {
        noMatch.style.display = (visibleCount === 0) ? 'block' : 'none';
    }
}
```

After the loop, update the visible-count display, and show the "no results" message only if zero cards ended up visible.

6. Wiring Up the Filter Buttons

```
filterButtons.forEach(function(button) {
    button.addEventListener('click', function() {
        filterButtons.forEach(function(b) {
            b.classList.remove('cmp-coursecards__filter--active');
        });
        button.classList.add('cmp-coursecards__filter--active');
        currentCategory = button.getAttribute('data-filter') || 'all';
        applyFilters();
    });
});
```

For every filter button, attach a click listener. When clicked: (1) remove the active class from *all* filter buttons, (2) add it to the one that was clicked so it appears selected, (3) read this button's **data-filter** attribute into `currentCategory`, and (4) re-run `applyFilters()` to update the cards.

7. Wiring Up the Dropdown

```
if (limitSelect) {
    limitSelect.addEventListener('change', function() {
        currentLimit = limitSelect.value;
        applyFilters();
    });
}
```

If the limit dropdown exists, listen for **change** events. When the user picks a new value, update `currentLimit` and re-apply the filters.

8. Initial Render

```
applyFilters();
```

Run `applyFilters()` once on load so the cards display correctly from the start, respecting the dropdown's default value.

9. The DOM-Ready Guard

```
if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', initCourseCardsFilter);
} else {
    initCourseCardsFilter();
}
```

This handles *when* to run the init function. If the document is still parsing (`'loading'`), wait for **DOMContentLoaded** so the cards exist in the DOM before querying. Otherwise, run immediately. This makes the script safe to include in either `<head>` or at the end of `<body>`.

10. Overall Flow in One Sentence

For each course-cards component on the page, set up independent state, listen to filter button clicks and dropdown changes, and re-run a single function that hides or shows cards based on the current category and visible-count limit.